Laporan Praktikum Transformasi Geometri pada Citra Digital



SYAHRIZAL HAMDALA

20240040310

1. Pendahuluan

Transformasi geometri adalah operasi pada citra digital yang mengubah posisi, ukuran, atau orientasi piksel tanpa mengubah warna dasar objek. Operasi ini banyak dipakai pada pengolahan citra, misalnya untuk koreksi posisi, penyesuaian ukuran, augmentasi data pada machine learning, dan penyelarasan citra.

Pada praktikum ini, sebuah citra berformat PNG (`#Kabinet.png`) ditransformasikan dengan lima jenis operasi: translasi, penskalaan, rotasi, flip horizontal, dan flip vertikal. Seluruh hasil ditampilkan dalam satu gambar berukuran 3×3 agar mudah dibandingkan dengan citra aslinya.

2. Tujuan

1. Memahami konsep transformasi geometri pada citra digital.
2. Mengimplementasikan translation, scaling, rotation, horizontal flip, dan vertical flip menggunakan OpenCV.
3. Mengamati pengaruh setiap transformasi terhadap posisi, ukuran, dan orientasi citra.

3. Landasan Teori

Translation (translasi) menggeser citra sejauh `tx` pada sumbu x dan `ty` pada sumbu y. Operasi ini memakai matriks afine:

```
| 1   0   tx |
| 0   1   ty |
```

Scaling (penskalaan) mengubah ukuran citra dengan faktor `fx` dan `fy`. Faktor kurang dari 1 memperkecil citra, sedangkan faktor lebih dari 1 memperbesarnya. Karena piksel baru harus dihitung, diperlukan interpolasi; pada praktikum ini dipakai `INTER_LINEAR` (bilinear).

Rotation (rotasi) memutar citra terhadap suatu titik pusat dengan sudut tertentu. OpenCV menyediakan `cv2.getRotationMatrix2D`, dan sudut positif berarti putaran berlawanan arah jarum jam. Agar seluruh citra tetap terlihat setelah diputar, ukuran kanvas baru dihitung dengan rumus:

```
lebar_baru  = tinggi × |sin θ| + lebar × |cos θ|
tinggi_baru = tinggi × |cos θ| + lebar × |sin θ|
```

Flip (pencerminan) membalik citra terhadap sumbu tertentu. Horizontal flip mencerminkan citra kiri-kanan (`cv2.flip(img, 1)`), sedangkan vertical flip mencerminkan citra atas-bawah (`cv2.flip(img, 0)`).

4. Alat dan Bahan

- Python 3 dengan pustaka OpenCV (`cv2`), NumPy, dan Matplotlib
- Citra masukan: `#Kabinet.png`
- Lingkungan pemrograman: Jupyter Notebook / Google Colab / editor Python

5. Implementasi Program

Berikut kode lengkap yang digunakan.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

nama_file = "#Kabinet.png"

img_bgr = cv2.imread(nama_file)
if img_bgr is None:
    raise FileNotFoundError(f"Citra '{nama_file}' tidak
ditemukan atau tidak dapat dibaca.")

img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)


def translation(image, tx, ty):
    matrix = np.float32([[1, 0, tx], [0, 1, ty]])
    ukuran = (image.shape[1], image.shape[0])
    return cv2.warpAffine(image, matrix, ukuran,
borderValue=(255, 255, 255))


def scaling(image, faktor):
    return cv2.resize(
        image,
        None,
        fx=faktor,
        fy=faktor,
        interpolation=cv2.INTER_LINEAR,
    )


def rotation(image, sudut):
    tinggi, lebar = image.shape[:2]
    pusat = (lebar / 2, tinggi / 2)
    matrix = cv2.getRotationMatrix2D(pusat, sudut, 1.0)

    cos = abs(matrix[0, 0])
    sin = abs(matrix[0, 1])
    lebar_baru = int(tinggi * sin + lebar * cos)
    tinggi_baru = int(tinggi * cos + lebar * sin)

    matrix[0, 2] += lebar_baru / 2 - pusat[0]
    matrix[1, 2] += tinggi_baru / 2 - pusat[1]
```

```
    return cv2.warpAffine(
        image,
        matrix,
        (lebar_baru, tinggi_baru),
        borderValue=(255, 255, 255),
    )


def horizontal_flip(image):
    return cv2.flip(image, 1)


def vertical_flip(image):
    return cv2.flip(image, 0)


hasil = [
    ("Citra Asli", img),
    ("Translation (tx=100, ty=50)", translation(img, 100,
50)),
    ("Scaling 0.5x", scaling(img, 0.5)),
    ("Scaling 2x", scaling(img, 2.0)),
    ("Rotation 45°", rotation(img, 45)),
    ("Rotation 90°", rotation(img, 90)),
    ("Rotation 180°", rotation(img, 180)),
    ("Horizontal Flip", horizontal_flip(img)),
    ("Vertical Flip", vertical_flip(img)),
]

print(f"Citra Asli: {img.shape}")
for judul, citra in hasil[1:]:
    print(f"{judul}: {citra.shape}")

fig, axes = plt.subplots(3, 3, figsize=(15, 15))
for ax, (judul, citra) in zip(axes.ravel(), hasil):
    ax.imshow(citra)
    ax.set_title(judul)
    ax.axis("off")

plt.tight_layout()
plt.savefig("hasil_transformasi.png", dpi=120,
bbox_inches="tight")
plt.show()
```

Penjelasan Kode

1. **Membaca citra.** `cv2.imread` membaca file dalam format BGR. Jika file tidak ditemukan, program menghentikan proses dengan pesan galat. Citra kemudian dikonversi ke RGB dengan `cv2.cvtColor` agar warna tampil benar di Matplotlib.
2. **Fungsi `translation`.** Membuat matriks translasi 2×3 lalu menerapkannya dengan `cv2.warpAffine`. Ukuran keluaran sama dengan citra asli, dan area kosong diisi warna putih (`borderValue=(255, 255, 255)`).
3. **Fungsi `scaling`.** Memakai `cv2.resize` dengan faktor `fx` dan `fy` yang sama sehingga rasio aspek tetap terjaga.
4. **Fungsi `rotation`.** Membuat matriks rotasi terhadap titik pusat citra, menghitung ukuran kanvas baru agar tidak ada bagian yang terpotong, lalu menggeser matriks agar citra tetap berada di tengah kanvas baru.
5. **Fungsi flip.** `cv2.flip` dengan parameter `1` untuk pencerminan horizontal dan `0` untuk pencerminan vertikal.
6. **Visualisasi.** Sembilan hasil ditampilkan pada grid 3×3, lalu disimpan sebagai `hasil_transformasi.png`.

6. Hasil dan Pembahasan

Gambar 1. Hasil seluruh transformasi (`hasil_transformasi.png`). Sisipkan gambar tersebut di bawah ini.

[Tempatkan gambar hasil_transformasi.png di sini]

Tabel berikut merangkum pengamatan dari setiap transformasi pada Gambar 1.

No	Transformasi	Parameter	Hasil Pengamatan
1	Citra Asli	-	Potret seorang pria berjas merah marun dengan latar kantor yang buram. Teks "Koordinator Syahrizal Hamdala" berada di kanan bawah dan logo kabinet di kiri atas.
2	Translation	tx = 100, ty = 50	Seluruh isi citra bergeser 100 piksel ke kanan dan 50 piksel ke bawah. Bagian kanan dan bawah terpotong (teks nama dan ikon di tepi bawah terlihat tidak utuh), sedangkan sisi kiri dan atas terisi warna putih.
3	Scaling	0,5×	Citra diperkecil menjadi setengah ukuran asli pada tiap sisi, dengan isi dan rasio aspek tetap sama.
4	Scaling	2×	Citra diperbesar dua kali lipat pada tiap sisi. Detail tetap terbaca, tetapi tepi dan teks kecil terlihat sedikit lebih halus karena interpolasi bilinear.
5	Rotation	45°	Citra berputar berlawanan arah jarum jam. Kanvas membesar menjadi bentuk persegi yang lebih besar sehingga seluruh citra tetap utuh, dan sudut-sudut kanvas diisi warna putih.
6	Rotation	90°	Citra berputar berlawanan arah jarum jam sehingga teks terbaca dari bawah ke atas. Lebar dan tinggi citra saling bertukar, dan tidak ada area putih tambahan.
7	Rotation	180°	Citra terbalik total. Posisi objek berpindah diagonal, dari kiri-atas ke kanan-bawah dan sebaliknya.

8	Horizontal Flip	sumbu vertikal	Citra tercermin kiri-kanan. Teks menjadi terbalik seperti bayangan cermin, dan logo berpindah dari kiri atas ke kanan atas.
9	Vertical Flip	sumbu horizontal	Citra tercermin atas-bawah. Objek tampak terbalik secara vertikal, tetapi posisi kiri-kanan tetap.

Pembahasan

- **Translasi** tidak mengubah ukuran kanvas. Akibatnya, bagian citra yang bergeser keluar batas akan hilang, dan area yang kosong terisi warna latar (putih).
- **Scaling** mengubah dimensi citra, sedangkan isi gambar tetap proporsional. Hal ini dapat diperiksa melalui nilai `shape` yang dicetak program (misalnya tinggi dan lebar pada 0,5× menjadi setengah, dan pada 2× menjadi dua kali). Pada Gambar 1, ukuran tampilan tiap panel terlihat mirip karena Matplotlib menyesuaikan skala tampilan sehingga perbedaan ukuran sebenarnya hanya terlihat pada nilai `shape`.
- **Rotasi** pada sudut yang bukan kelipatan 90° (seperti 45°) memerlukan kanvas yang lebih besar agar tidak ada bagian yang terpotong. Rumus perhitungan ukuran kanvas baru pada fungsi `rotation` mengatasi hal ini. Untuk 90° dan 180°, ukuran citra tidak bertambah karena sudutnya sejajar dengan sumbu.
- **Rotasi 180° dan vertical flip** sama-sama membuat citra tampak terbalik, tetapi hasilnya berbeda. Rotasi 180° adalah putaran, sehingga tidak ada efek cermin, sedangkan vertical flip adalah pencerminan, yang hanya membalik sumbu vertikal sehingga sisi kiri dan kanan tidak tertukar.
- **Flip** tidak mengubah ukuran maupun kualitas citra karena hanya menyusun ulang posisi piksel tanpa interpolasi.

7. Kesimpulan

1. Transformasi geometri dapat diimplementasikan dengan mudah menggunakan OpenCV melalui `cv2.warpAffine`, `cv2.resize`, dan `cv2.flip`.
2. Translasi menggeser posisi citra tanpa mengubah ukuran kanvas, sehingga sebagian isi dapat terpotong.
3. Scaling mengubah dimensi citra dengan rasio aspek terjaga dan memerlukan interpolasi untuk menghasilkan piksel baru.
4. Rotasi memerlukan penyesuaian ukuran kanvas agar seluruh isi citra tetap terlihat, terutama untuk sudut yang bukan kelipatan 90°.
5. Flip menghasilkan citra cermin tanpa kehilangan informasi, dan berbeda dari rotasi 180° meskipun sama-sama tampak terbalik.

8. Daftar Pustaka

- OpenCV Documentation. *Geometric Transformations of Images*. https://docs.opencv.org
- Gonzalez, R. C., & Woods, R. E. *Digital Image Processing*. Pearson.